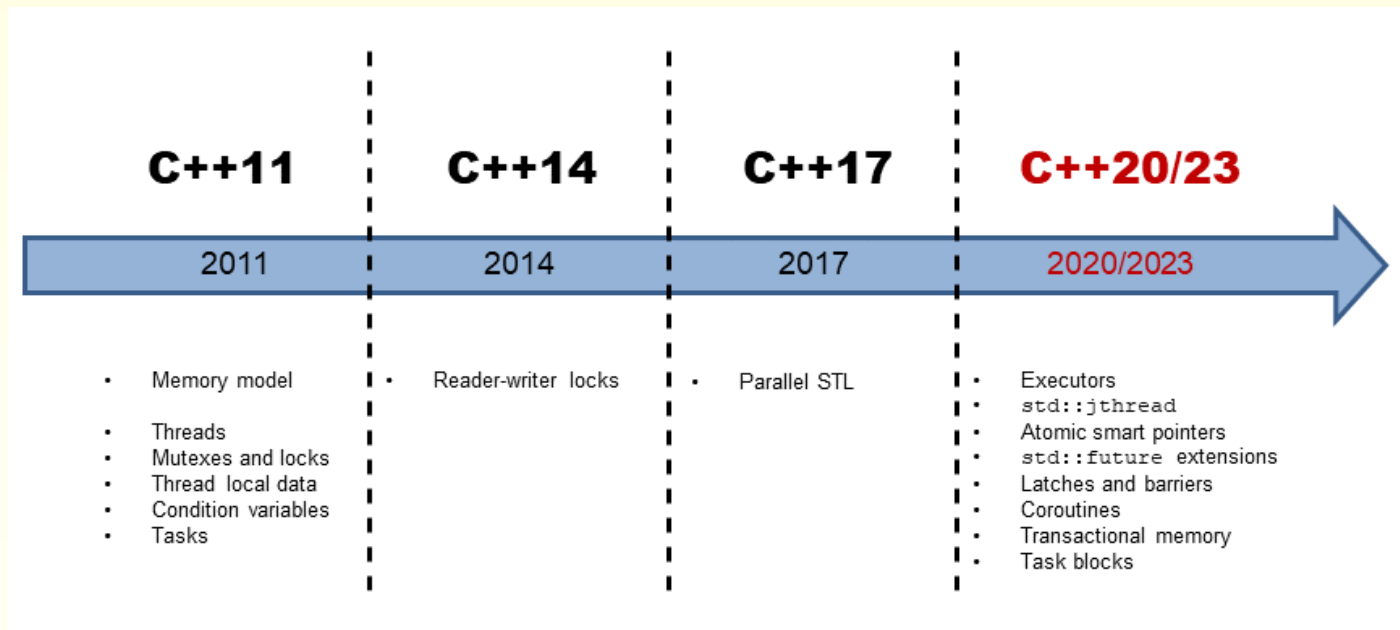


Lecture 5–2 :

C++ Multithreaded Programming

C++ Multi-Threading

- C++11
 - First C++ standard that introduced threads, mutexes (locks), conditional variables etc
- C++17
 - Parallel STL: addition of parallel algorithms in STL



C++ Thread Library

- include header file **<thread>**
- Create and starts a new thread using **std::thread**
 - Syntax: **thread(func, args...)**
 - **func** will be called in a new thread
 - The new thread will terminate when **func** returns
 - All parameters passed to **func** are passed by value
 - For pass by reference, wrap them in **std::ref**
- 4 ways to create a thread in C++ (see the example code)
 - using function pointer
 - using functor (function object)
 - using class member function
 - using lambda expression

C++ Thread Library

- Call the member function **join()** to wait for a thread to finish
- Call the member function **detach()** to run independently.
 - No join: join is not allowed after calling detach()
 - If thread object is destructed without calling detach(), error might occur.
 - After a call to this function, the thread object becomes non-joinable and can be destroyed safely
- The parent thread should call **join()** or **detach()**
- Include header file <mutex> to use **std::mutex**
 - **ex) std::mutex m;**
 m.lock();
 m.unlock();
 - **std::lock_guard:** locks a supplied mutex on construction and unlocks it on destruction (simple exclusive lock)

C++ Thread Library

- **std::atomic** for atomic variables
 - operations are atomic
 - behave as if it is inside a mutex-protected critical section
 - usually faster than mutex lock (HW supported)
 - Example

```
std::atomic<int> ai(0); //initialize ai to 0
ai = 10; //atomically set ai to 10
std::cout << ai; //atomically read ai's value
++ai; //atomically increment ai to 11
--ai; //atomically decrement ai to 10
```

- Basic example codes for thread creation and mutual exclusion (lock) are available on our class webpage

Parallel STL

- C++ STL algorithms with support for execution policies (C++17 or later)

- Execution Policy

- `std::function(execution_policy, ... other_arguments ...);`
- `ex) std::sort(std::execution::par, v.begin(), v.end());`
- `std::execution::seq`
 - 1 thread, sequential
- `std::execution::par`
 - multiple threads
- `std::execution::par_unseq`
 - multiple threads, SIMD (vectorization) possible
- `std::execution::unseq`
 - 1 thread, SIMD (vectorization) possible